*Last updated on **May 6, 2026***

# Build gen AI apps on Databricks

This page provides an overview of tools for building, deploying, and managing generative AI (gen AI) apps on Databricks.

**Get started: no-code GenAI**

Try AI Playground for UI-based testing and prototyping.

**Get started: MLflow 3 for GenAI**

Try MLflow for GenAI tracing, evaluation, and human feedback.

## Serve and query gen AI large language models (LLMs)

Serve a curated set of gen AI models from LLM providers such as OpenAI and Anthropic and make them available through secure, scalable APIs.

| Feature | Description |
|---|---|
| Foundation Models | Serve gen AI models, including open source and third-party models such as **Meta Llama**, **Anthropic Claude**, **OpenAI GPT**, and more. |

## Build and deploy enterprise-grade AI agents

Build and deploy your own agents, including tool-calling agents, retrieval-augmented generation apps, and multi-agent systems.

| Feature | Description |
| --- | --- |
| AI Playground (no code) | Prototype and test AI agents in a no-code environment. Quickly experiment with agent behaviors and tool integrations before generating code for deployment. |
| Knowledge Assistant | Build and optimize domain-specific AI chatbots using an intuitive interface. |
| Build custom agents | Author, deploy, and evaluate agents using Python. Supports agents written with any authoring library. Integrated with MLflow Tracing. Iterate quickly using Databricks Apps. |
| AI agent tools | Create agent tools to query structured and unstructured data, run code, or connect to external service APIs. |
| MCP (Model Context Protocol) | Standardize how agents connect to data and tools with a secure, consistent interface. |

# Evaluate, debug, and optimize agents

Track agent performance, collect feedback, and drive quality improvements with evaluation and tracing tools.

| Feature | Description |
| --- | --- |
| MLflow Tracing | Use MLflow Tracing for end-to-end observability. Log every step your agent takes to debug, monitor, and audit agent behavior in development and production. |

| Feature | Description |
| --- | --- |
| Agent Evaluation | Use Agent Evaluation and MLflow to measure quality, cost, and latency. Collect feedback from stakeholders and subject matter experts through built-in review apps and use LLM judges to identify and resolve quality issues. |
| Monitor agents | Use the same evaluation configuration (LLM judges and custom metrics) in offline evaluation and online monitoring. |

*Last updated on **May 6, 2026***

# Create an AI agent

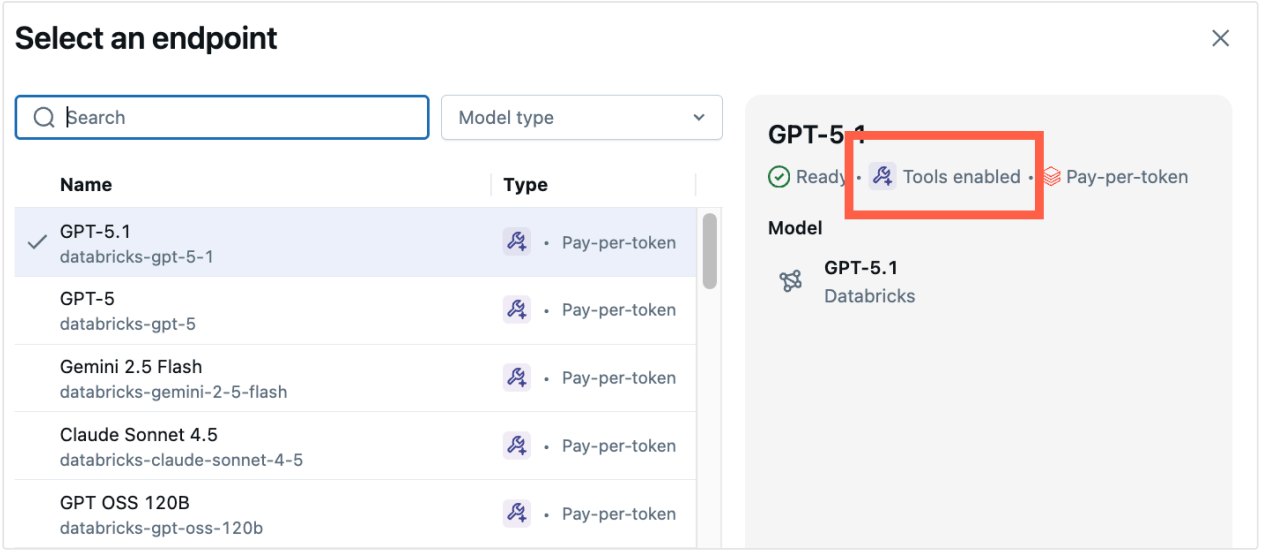
This article introduces the process of creating AI agents on Databricks and outlines the available methods for creating agents.

To learn more about agents, see Agent system design patterns.

## Prototype agents with AI Playground

The AI Playground is the easiest way to create an agent on Databricks. AI Playground lets you select from various LLMs and quickly add tools to the LLM using a low-code UI. You can then chat with the agent to test its responses and then export the agent to code for deployment or further development.

See Get started: Query LLMs and prototype AI agents with no code.




## Automatically build an agent with Knowledge Assistant

Knowledge Assistant provides a streamlined approach to build and optimize domain-specific, question-and-answer chatbots over your documents and improve quality based on natural language feedback from your subject matter experts.

Knowledge Assistant has a fully managed approach that is a good place to start before diving into more bespoke agents.

# Code a custom agent

Agent Framework and MLflow has tools to help you author enterprise-ready agents in Python.

Databricks supports authoring agents using third-party agent authoring libraries like LangGraph/LangChain, OpenAI, LlamaIndex, or custom Python implementations.

To get started quickly, see Get started with AI agents. For more details on authoring agents with different frameworks and advanced features, see Author an AI agent and deploy it on Databricks Apps.

# Understand model signatures to ensure compatibility with Databricks features

Databricks uses MLflow Model Signatures to define agents' input and output schema. Product features like the AI Playground assume that your agent has one of a set of supported model signatures.

If you follow the recommended approach to authoring agents using the ResponsesAgent interface, MLflow will automatically infer a signature for your agent that is compatible with Databricks product features.

*Last updated on **May 8, 2026***

# AI agent tools

AI agent tools give your agents practical capabilities beyond text generation, like searching documents, querying databases, calling REST APIs, or running custom code.

Use pre-configured managed MCP servers for immediate access to Databricks data, use external MCP servers to connect to third-party APIs, or build custom tools for specialized business logic.

## Tools and examples

Common tool patterns and implementation examples:

### Work with structured data

Read structured data in Unity Catalog tables.

### Retrieve unstructured data

Connect agents to vector search indexes to query unstructured data.

### Code interpreter tools

Let agents run Python code dynamically using the built-in `system.ai.python_exec` tool.

### AI tools using Unity Catalog functions

Create tools using Unity Catalog functions. (Databricks recommends using MCP tools instead for most new use cases.)

## MCP servers

Use MCP servers to give agents access to governed Databricks data and third-party APIs:

### Connect agents to external services

Use managed OAuth, the UC connections proxy, or external Rest APIs to connect agents to third-party APIs.

### Use external MCP servers in agents

Call external MCP servers from agent code through Databricks-managed proxies. To install an external MCP server, see Install an external MCP server.

### Use custom MCP servers in agents

Connect agent code to a custom MCP server hosted as a Databricks app. To host a custom MCP server, see Host a custom MCP server.

*Last updated on **May 11, 2026***

# Connect agents to structured data

AI agents often need to query or manipulate structured data to answer questions, update records, or create data pipelines.

Databricks provides multiple approaches for connecting agents to structured data in Unity Catalog tables and external data stores. Use pre-configured MCP servers for immediate access to Genie Spaces and SQL warehouses, or build custom tools for specialized workflows.

This page shows how to:

- Query data in Unity Catalog tables
- Use Genie in advanced multi-agent systems
- Use UC functions to run deterministic, repeatable queries

## Query data in Unity Catalog tables

If your agent needs to query data in Unity Catalog tables, Databricks recommends using Genie spaces. A Genie space is a collection of up to 25 Unity Catalog tables that Genie can keep in context and query using natural language. Agents can access the Genie space using a pre-configured MCP URL.

To connect to a Genie space:

1. Create a Genie space with the tables you want to query and share the space with the users, or service principals, that must access it. See Create and manage a Genie Space.

2. Create an agent and connect it to the pre-configured managed MCP URL for the space: `https://<workspace-hostname>/api/2.0/mcp/genie/{genie_space_id}`.

> **NOTE**
>
> The managed MCP server for Genie invokes Genie as an MCP tool, which means history isn't passed when invoking Genie APIs.

# Add a Genie space tool to your agent

The following examples show how to connect your agent to a Genie space MCP server. Replace `<genie-space-id>` with the ID of your Genie space.

OpenAI Agents SDK (Apps) | LangGraph (Apps) | Model Serving

Python

```python
from agents import Agent, Runner
from databricks.sdk import WorkspaceClient
from databricks_openai.agents import McpServer

workspace_client = WorkspaceClient()
host = workspace_client.config.host

async with McpServer(
    url=f"{host}/api/2.0/mcp/genie/<genie-space-id>",
    name="genie-space",
    workspace_client=workspace_client,
) as genie_server:
    agent = Agent(
        name="Data analyst agent",
        instructions="You are a data analyst. Use the Genie tool to query
structured data and answer questions.",
        model="databricks-claude-sonnet-4-5",
        mcp_servers=[genie_server],
    )
    result = await Runner.run(agent, "What were the top 10 customers by
revenue last quarter?")
    print(result.final_output)
```

Ask Genie

Grant the app access to the Genie space in `databricks.yml`:

YAML

```yaml
resources:
  apps:
    my_agent_app:
      resources:
        - name: 'my_genie_space'
          genie_space:
            space_id: '<genie-space-id>'
            permission: 'CAN_RUN'
```

# Genie multi-agent system

> **PREVIEW**
>
> This feature is in Public Preview.

For advanced, multi-agent systems, you can also use Genie as an agent rather than integrating it using MCP. When you call Genie as an agent, you can deterministically pass in existing conversation context to Genie.

For a code-first approach, see Use Genie in multi-agent systems (Model Serving). For a UI-first approach, see Use Supervisor Agent to create a coordinated multi-agent system.

# Query data using Unity Catalog SQL function tool

Create a structured retrieval tool using Unity Catalog SQL functions when the query is known ahead of time and the agent provides the parameters.

The following example creates a Unity Catalog function called `lookup_customer_info`, which allows an AI agent to retrieve structured data from a hypothetical `customer_data` table.

Run the following code in a SQL editor.

SQL

```
CREATE OR REPLACE FUNCTION main.default.lookup_customer_info(
  customer_name_input STRING COMMENT 'Name of the customer whose info to
look up'
)
RETURNS STRING
COMMENT 'Returns metadata about a particular customer, given the
customer''s name, including the customer''s email and ID. The
customer ID can be used for other queries.'
RETURN SELECT CONCAT(
    'Customer ID: ', customer_id, ', ',
    'Customer Email: ', customer_email
  )
  FROM main.default.customer_data
  WHERE customer_name = customer_name_input
  LIMIT 1;
```

After you create a Unity Catalog tool, add it to your agent. See Create a Unity Catalog function tool.

*Last updated on **May 12, 2026***

# Connect agents to unstructured data

AI agents often need to query unstructured data like document collections, knowledge bases, or text corpora to answer questions and provide context-aware responses.

Databricks provides multiple approaches for connecting agents to unstructured data in Vector Search indexes and external vector stores. Use pre-configured MCP servers for immediate access to Databricks Vector Search indexes, develop retriever tools locally with AI Bridge packages, or build custom retriever functions for specialized workflows.

## Query a Databricks Vector Search index using MCP

If your agent needs to query a Databricks Vector Search index, use the Databricks-managed MCP server. Before you start, create a Vector Search index using Databricks-managed embeddings. See Create vector search endpoints and indexes.

The managed MCP URL for Vector Search is: `https://<workspace-hostname>/api/2.0/mcp/vector-search/{catalog}/{schema}/{index_name}`.

The following examples show how to connect your agent to a Vector Search index. Replace `<catalog>`, `<schema>`, and `<index-name>` with the names of your Vector Search index.

**OpenAI Agents SDK (Apps)** | **LangGraph (Apps)** | **Model Serving**

Python

```python
from agents import Agent, Runner
from databricks.sdk import WorkspaceClient
from databricks_openai.agents import McpServer

workspace_client = WorkspaceClient()

async with McpServer.from_vector_search(
    catalog="<catalog>",
    schema="<schema>",
    index_name="<index-name>",
    workspace_client=workspace_client,
    name="vector-search",
) as vs_server:
    agent = Agent(
        name="Research assistant",
        instructions="You are a research assistant. Use the vector search tool to find relevant documents and answer questions.",
        model="databricks-claude-sonnet-4-5",
        mcp_servers=[vs_server],
    )
    result = await Runner.run(agent, "What is the return policy?")
    print(result.final_output)
```

Grant the app access to the Vector Search index in `databricks.yml`:

YAML

```yaml
resources:
  apps:
    my_agent_app:
      resources:
        - name: 'my_vector_index'
          uc_securable:
            securable_full_name: '<catalog>.<schema>.<index-name>'
            securable_type: 'TABLE'
            permission: 'SELECT'
```

# Other approaches

Query a vector search index outside Databricks

Develop a local retriever

Query Databricks Vector Search using UC functions (deprecated)

# Add tracing to a retriever tool

Add MLflow tracing to monitor and debug your retriever. Tracing lets you view inputs, outputs, and metadata for each step of execution.

The previous example adds the @mlflow.trace decorator to both the `__call__` and parsing methods. The decorator creates a span that starts when the function is invoked and ends when it returns. MLflow automatically records the function's input and output and any exceptions raised.

> **NOTE**
>
> LangChain, LlamaIndex, and OpenAI library users can use MLflow auto logging in addition to manually defining traces with the decorator. See Add traces to applications: automatic and manual tracing.

Python

```
import mlflow
from mlflow.entities import Document

# This code snippet has been truncated for brevity. See the full
retriever example above.
class VectorSearchRetriever:
  ...

  # Create a RETRIEVER span. The span name must match the retriever
schema name.
  @mlflow.trace(span_type="RETRIEVER", name="vector_search")
  def __call__(...) -> List[Document]:
    ...

  # Create a PARSER span.
  @mlflow.trace(span_type="PARSER")
```

Ask Genie

```
def parse_results(...) -> List[Document]:
  ...
```

To verify downstream applications such as Agent Evaluation and the AI Playground render the retriever trace correctly, make sure the decorator meets the following requirements:

- Use the MLflow retriever span schema and verify the function returns a List[Document] object.
- The trace name and the `retriever_schema` name must match to configure the trace correctly. See the following section to learn how to set the retriever schema.

## Set retriever schema to verify MLflow compatibility

If the trace returned from the retriever or `span_type="RETRIEVER"` does not conform to MLflow's standard retriever schema, you must manually map the returned schema to MLflow's expected fields. This verifies that MLflow can properly trace your retriever and render traces in downstream applications.

To set the retriever schema manually:

1. Call mlflow.models.set_retriever_schema when you define your agent. Use `set_retriever_schema` to map the column names in the returned table to MLflow's expected fields such as `primary_key`, `text_column`, and `doc_uri`.

   Python

   ```
   # Define the retriever's schema by providing your column names
   mlflow.models.set_retriever_schema(
     name="vector_search",
     primary_key="chunk_id",
     text_column="text_column",
     doc_uri="doc_uri"
     # other_columns=["column1", "column2"],
   )
   ```

2. Specify additional columns in your retriever's schema by providing a list of column names with the `other_columns` field.

3. If you have multiple retrievers, you can define multiple schemas by using unique names for each retriever schema.

The retriever schema set during agent creation affects downstream applications and workflows, such as the review app and evaluation sets. Specifically, the `doc_uri` column serves as the primary identifier for documents returned by the retriever.

- The **review app** displays the `doc_uri` to help reviewers assess responses and trace document origins. See Review App UI.
- **Evaluation sets** use `doc_uri` to compare retriever results against predefined evaluation datasets to determine the retriever's recall and precision. See Evaluation sets (MLflow 2).

# Read files from a Unity Catalog volume

If your agent needs to read unstructured files (text documents, reports, configuration files, etc.) stored in a Unity Catalog volume, you can create tools that use the Databricks SDK Files API to list and read files directly.

The following examples create two tools your agent can use:

- **`list_volume_files`**: Lists files and directories in the volume.
- **`read_volume_file`**: Reads the contents of a text file from the volume.

LangChain/LangGraph | OpenAI

Install the latest version of `databricks-langchain` that includes Databricks AI Bridge.

Bash

```
%pip install --upgrade databricks-langchain
```

Python

```
from databricks.sdk import WorkspaceClient
from langchain_core.tools import tool
```

```
VOLUME = "<catalog>.<schema>.<volume>"  # TODO: Replace with your volume
w = WorkspaceClient()


@tool
def list_volume_files(directory: str = "") -> str:
    """Lists files and directories in the Unity Catalog volume.
    Provide a relative directory path, or leave empty to list the volume
root."""
    base = f"/Volumes/{VOLUME.replace('.', '/')}"
    path = f"{base}/{directory.lstrip('/')}" if directory else base
    entries = []
    for f in w.files.list_directory_contents(path):
        kind = "dir" if f.is_directory else "file"
        size = f" ({f.file_size} bytes)" if not f.is_directory else ""
        entries.append(f"  [{kind}] {f.name}{size}")
    return "\n".join(entries) if entries else "No files found."


@tool
def read_volume_file(file_path: str) -> str:
    """Reads a text file from the Unity Catalog volume.
    Provide the path relative to the volume root, for example
'reports/q1_summary.txt'."""
    base = f"/Volumes/{VOLUME.replace('.', '/')}"
    full_path = f"{base}/{file_path.lstrip('/')}"
    resp = w.files.download(full_path)
    return resp.contents.read().decode("utf-8")
```

Bind the tools to an LLM and run a tool-calling loop:

Python

```
from databricks_langchain import ChatDatabricks
from langchain_core.messages import HumanMessage, ToolMessage

llm = ChatDatabricks(endpoint="databricks-claude-sonnet-4-5")
llm_with_tools = llm.bind_tools([list_volume_files, read_volume_file])

messages = [HumanMessage(content="What files are in the volume? Can you
read about_databricks.txt and summarize it in 2 sentences?")]
tool_map = {"list_volume_files": list_volume_files, "read_volume_file":
read_volume_file}

for _ in range(5):  # max iterations
    response = llm_with_tools.invoke(messages)
```

```
    messages.append(response)
    if not response.tool_calls:
        break
    for tc in response.tool_calls:
        result = tool_map[tc["name"]].invoke(tc["args"])
        messages.append(ToolMessage(content=result,
tool_call_id=tc["id"]))

print(response.content)
```

**this page**

**as this page helpful?**

Yes | No

Send feedback

Ask Genie

*Last updated on **May 6, 2026***

# Python code interpreter agent tool

Databricks provides `system.ai.python_exec`, a built-in Unity Catalog function that lets AI agents dynamically run Python code written by the agent, provided by a user, or retrieved from a codebase. It is available by default and can be used directly in a SQL query:

SQL

```
SELECT python_exec("""
import random
numbers = [random.random() for _ in range(10)]
print(numbers)
""")
```

To learn more about agent tools, see AI agent tools.

# Add the code interpreter to your agent

To add `python_exec` to your agent, connect to the managed MCP server for the `system.ai` Unity Catalog schema. The code interpreter is available as a pre-configured MCP tool at `https://<workspace-hostname>/api/2.0/mcp/functions/system/ai/python_exec`.

OpenAI Agents SDK (Apps) | LangGraph (Apps) | Model Serving

Python

Ask Genie

```python
from agents import Agent, Runner
from databricks.sdk import WorkspaceClient
from databricks_openai.agents import McpServer

# WorkspaceClient picks up credentials from the environment (Databricks
Apps, notebook, CLI)
workspace_client = WorkspaceClient()
host = workspace_client.config.host

# The context manager manages the MCP connection lifecycle and ensures
cleanup on exit.
# from_uc_function constructs the endpoint URL from UC identifiers and
wires in auth
# from workspace_client, avoiding hardcoded URLs and manual token
handling.
async with McpServer.from_uc_function(
    catalog="system",
    schema="ai",
    function_name="python_exec",
    workspace_client=workspace_client,
    name="code-interpreter",
) as code_interpreter:
    agent = Agent(
        name="Coding agent",
        instructions="You are a helpful coding assistant. Use the
python_exec tool to run code.",
        model="databricks-claude-sonnet-4-5",
        mcp_servers=[code_interpreter],
    )
    result = await Runner.run(agent, "Calculate the first 10 Fibonacci
numbers")
    print(result.final_output)
```

Grant the app access to the function in `databricks.yml`:

YAML

```yaml
resources:
  apps:
    my_agent_app:
      resources:
        - name: 'python_exec'
          uc_securable:
            securable_full_name: 'system.ai.python_exec'
            securable_type: 'FUNCTION'
            permission: 'EXECUTE'
```

# Next steps

- AI agent tools - Explore other tool types for your agent.
- Create AI agent tools using Unity Catalog functions - Create custom tools using Unity Catalog functions.
- Use Databricks managed MCP servers - Learn more about managed MCP servers on Databricks.

*Last updated on **May 9, 2026***

# Connect agents to external services

**PREVIEW**

This feature is in Public Preview.

Connect your AI agents to external applications like Slack, Google Calendar, or any service with an API. Databricks provides several approaches depending on whether the external service has an MCP server, whether you need per-user authentication, or whether you prefer to call APIs directly from agent code. All approaches rely on a Unity Catalog HTTP connection to securely manage credentials and govern access to external services.

| Approach | Recommended use case |
| --- | --- |
| External MCP servers | Use this approach for services that publish an MCP server. It provides automatic tool discovery and works with standard SDKs. |
| Managed OAuth | Use this approach for Google Drive or SharePoint integrations. Databricks manages the OAuth credentials, so no app registration is required. |
| UC connections proxy | Use this approach to make direct REST API calls from agent code using the external service's own client SDK. |
| UC function tools | Use this approach for SQL-based tool definitions that wrap the `http_request()` function. |

# Requirements

- A Unity Catalog HTTP connection for your external application. Unity Catalog connections provide secure, governed credential management and enable multiple authentication methods, including OAuth 2.0 user-to-machine and machine-to-machine authentication.

# External MCP servers

If the external service has an MCP server available, Databricks recommends connecting through external MCP servers. MCP servers provide automatic tool discovery, simplified integration, and per-user authentication.

- See Install an external MCP server for installation and authentication setup.
- See Use external MCP servers in agents for code examples per agent framework (OpenAI Agents SDK, LangGraph, Model Serving).

# Managed OAuth

Databricks offers managed OAuth flows for select API tool providers. You don't need to register your own OAuth app or manage credentials. Databricks recommends Managed OAuth for development and testing. If production use cases require generating custom OAuth credentials, see the providers' documentation for more information.

The following integrations use Databricks-managed OAuth credentials stored securely in the backend.

| Provider | Configuration notes | Supported scopes |
| --- | --- | --- |
| **Google Drive API** | None | `https://www.googleapis.com/auth/drive.readonly` `offline_access` |

| Provider | Configuration notes | Supported scopes |
| --- | --- | --- |
| | | |
| **SharePoint API** | None | `https://graph.microsoft.com/Sites.Read.All offline_access openid profile` |

To set up managed OAuth, create an HTTP connection with the **OAuth User to Machine Per User** auth type and select your provider from the **OAuth Provider** drop-down menu. For detailed steps, see Installation methods.

Each user is prompted to authorize with the provider on first use.

If needed, allowlist the following redirect URIs used by managed OAuth:

| Cloud | Redirect URI |
| --- | --- |
| AWS | `https://oregon.cloud.databricks.com/api/2.0/http/oauth/redirect` |
| Azure | `https://westus.azuredatabricks.net/api/2.0/http/oauth/redirect` |
| GCP | `https://us-central1.gcp.databricks.com/api/2.0/http/oauth/redirect` |

# UC connections proxy endpoint

Use the Unity Catalog connections proxy endpoint with the external service's own client SDK to call REST APIs directly from agent code. Point the SDK's base URL to the proxy endpoint and use your Databricks token as the API key. Databricks authenticates

the request and automatically injects the external service's credentials from the Unity Catalog connection. Your code never handles the external service's tokens directly.

**Permissions required:** `USE CONNECTION` on the connection object.

OpenAI | Slack | Generic HTTP

Use `DatabricksOpenAI` to route calls to external OpenAI through the Unity Catalog connections proxy. First, create a Unity Catalog HTTP connection using your OpenAI API key stored as a Databricks secret:

SQL

```sql
CREATE CONNECTION openai_connection TYPE HTTP
OPTIONS (
  host 'https://api.openai.com',
  base_path '/v1',
  bearer_token secret ('<secret-scope>', '<secret-key>')
);
```

Then install the `databricks-openai` package and use the proxy URL and workspace client in your agent code:

Bash

```bash
pip install databricks-openai
```

Python

```python
from databricks_openai import DatabricksOpenAI
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

client = DatabricksOpenAI(
    workspace_client=w,
    base_url=f"{w.config.host}/api/2.0/unity-
catalog/connections/openai_connection/proxy/",
)
```

```
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Hello!"}],
)
print(response.choices[0].message.content)
```

# UC function tools with HTTP connections

> **NOTE**
>
> Databricks recommends using MCP servers or the UC connections proxy for new integrations. UC function tools with `http_request` remain supported but are no longer the recommended approach.

You can create a Unity Catalog function that wraps `http_request()` to call external services. This approach is useful for SQL-based tool definitions. See Create AI agent tools using Unity Catalog functions for details on creating UC function tools.

The following example creates a Unity Catalog function tool that posts a message to Slack:

SQL

```
CREATE OR REPLACE FUNCTION main.default.slack_post_message(
  text STRING COMMENT 'message content'
)
RETURNS STRING
COMMENT 'Sends a Slack message by passing in the message and returns the
response received from the external service.'
RETURN (http_request(
  conn => 'test_sql_slack',
  method => 'POST',
  path => '/api/chat.postMessage',
  json => to_json(named_struct(
    'channel', "C032G2DAH3",
    'text', text
  ))
)).text
```

See CREATE FUNCTION (SQL and Python).

> **NOTE**
>
> SQL access with `http_request` is blocked for the User-to-Machine Per User and Dynamic Client Registration connection types. Use the Python Databricks SDK instead.

# Example notebooks

## Connect an agent to Slack

See Connect an AI agent to Slack.

## External connection tools

The following notebooks demonstrate creating AI agent tools that connect to Slack, OpenAI, and Azure AI search.

### Slack messaging agent tool

Open notebook in new tab

Copy link for import

```
%md
# Create agent tools for Slack

This notebook creates agent tools that interact with your Slack
workspace. These tools can be used to retrieve information, send
messages, or perform actions in your Slack workspace.

## Requirements

Before you begin, ensure that you have a token to authenticate to
your Slack workspace. If you use a bearer token for an app, all
actions will be performed as the app itself. However, if you use a
actions will be performed as the authenticated user.
Choose the token type that best fits your use case.
```

Expand notebook

### Microsoft Graph API agent tool

Ask Genie

Open notebook in new tab

Copy link for import

```
%md
# Create agent tools for Microsoft Graph API

This notebook creates agent tools to fetch data using Microsoft Graph
API's API.

To learn more about creating agent tools that connect to external
services, see Databricks documentation ([AWS]
(https://docs.databricks.com/generative-ai/agent-framework/external-
connection-tools.html) | [Azure]
(https://learn.microsoft.com/azure/databricks/generative-ai/agent-
framework/external-connection-tools)).
```

Expand notebook

## Requirements

## Azure AI Search agent tool

Open notebook in new tab

Copy link for import

# Create agent tools for Azure AI Search

This notebook creates agent tools that connect to and query Azure AI Search. These tools can retrieve information from your Azure AI Search index in Databricks.

To learn more about creating agent tools that connect to external services, see Databricks documentation (AWS (https://docs.databricks.com/generative-
ork/external-connection-tools.html) | Azure
(https://learn.microsoft.com/azure/databricks/generative-ai/agent-

Expand notebook

**this page**

**as this page helpful?**

Yes

No

Send feedback

Ask Genie

*Last updated on **May 6, 2026***

# Query an agent deployed on Databricks

Learn how to send requests to agents deployed to Databricks Apps or Model Serving endpoints. Databricks provides multiple query methods to fit different use cases and integration needs.

Select the query approach that best fits your use case:

| Method | Key benefits |
|---|---|
| Databricks OpenAI Client (Recommended) | Native integration, full feature support, streaming capabilities |
| REST API | OpenAI-compatible, language-agnostic, works with existing tools |
| AI Functions: `ai_query` | OpenAI-compatible, query legacy agents hosted on Model Serving endpoints only |

Databricks recommends the **Databricks OpenAI Client** for new applications. Choose the **REST API** when integrating with platforms that expect OpenAI-compatible endpoints.

## Databricks OpenAI Client (Recommended)

Databricks recommends that you use the DatabricksOpenAI Client to query a deployed agent. Depending on the API of your deployed agent, you will e responses or chat completions client:

Agents deployed to Apps | Agents on Model Serving

Use the following example for agents hosted on Databricks Apps following the `ResponsesAgent` interface, which is the recommended approach for building agents. You must use a Databricks OAuth token to query agents hosted on Databricks Apps.

Python

```
from databricks.sdk import WorkspaceClient
from databricks_openai import DatabricksOpenAI

input_msgs = [{"role": "user", "content": "What does Databricks do?"}]
app_name = "<agent-app-name>"  # TODO: update this with your app name

# The WorkspaceClient must be configured with OAuth authentication
# See: https://docs.databricks.com/aws/en/dev-tools/auth/oauth-u2m.html
w = WorkspaceClient()

client = DatabricksOpenAI(workspace_client=w)

# Run for non-streaming responses. Calls the "invoke" method
# Include the "apps/" prefix in the model name
response = client.responses.create(model=f"apps/{app_name}",
input=input_msgs)
print(response)

# Include stream=True for streaming responses. Calls the "stream" method
# Include the "apps/" prefix in the model name
streaming_response = client.responses.create(
    model=f"apps/{app_name}", input=input_msgs, stream=True
)
for chunk in streaming_response:
    print(chunk)
```

If you want to pass in `custom_inputs`, you can add them with the `extra_body` param:

Python

```
streaming_response = client.responses.create(
    model=f"apps/{app_name}",
    input=input_msgs,
    stream=True,
```

```
    extra_body={
        "custom_inputs": {"id": 5},
    },
)
for chunk in streaming_response:
    print(chunk)
```

To retrieve a trace ID from the response, include the `x-mlflow-return-trace-id` header using `extra_headers`. Then use MLflow `get_trace` to retrieve the full trace.

Python

```
response = client.responses.create(
    model=f"apps/{app_name}",
    input=input_msgs,
    extra_headers={"x-mlflow-return-trace-id": "true"},
)
trace_id = response.metadata["trace_id"]
trace = client.get_trace(trace_id)
```

# REST API

The Databricks REST API provides endpoints for models that are OpenAI-compatible. This allows you to use Databricks agents to serve applications that require OpenAI interfaces.

This approach is ideal for:

- Language-agnostic applications that use HTTP requests
- Integrating with third-party platforms that expect OpenAI-compatible APIs
- Migrating from OpenAI to Databricks with minimal code changes

Authenticate with the REST API using a Databricks OAuth token. Refer to the Databricks Authentication Documentation for more options and information.

**Agents deployed to Apps** | **Agents on Model Serving**

Use the following example for agents hosted on Databricks Apps following the `ResponsesAgent` interface, which is the recommended approach for building agents. You must use a Databricks OAuth token to query agents hosted on Databricks Apps.

Bash

```
curl --request POST \
  --url <app-url>.databricksapps.com/responses \
  --header 'Authorization: Bearer <OAuth token>' \
  --header 'content-type: application/json' \
  --data '{
    "input": [{ "role": "user", "content": "hi" }],
    "stream": true
  }'
```

If you want to pass in `custom_inputs`, you can add them to the request body:

Bash

```
curl --request POST \
  --url <app-url>.databricksapps.com/responses \
  --header 'Authorization: Bearer <OAuth token>' \
  --header 'content-type: application/json' \
  --data '{
    "input": [{ "role": "user", "content": "hi" }],
    "stream": true,
    "custom_inputs": { "id": 5 }
  }'
```

To retrieve a trace ID from the response, include the `x-mlflow-return-trace-id` header in your request. The response body includes a `metadata.trace_id` field containing the trace ID. For streaming requests, the trace ID is sent as a separate SSE event (`data: {"trace_id": "tr-..."}`) near the end of the stream. Then use MLflow `get_trace` to retrieve the full trace using the trace ID.

Bash

```
curl --request POST \
  --url <app-url>.databricksapps.com/responses \
  --header 'Authorization: Bearer <OAuth token>' \
  --header 'content-type: application/json' \
```

```
  --header 'x-mlflow-return-trace-id: true' \
  --data '{
    "input": [{ "role": "user", "content": "hi" }]
  }'
```

# AI Functions: `ai_query`

You can use `ai_query` to query a deployed agent hosted on model serving using SQL. See `ai_query` function for SQL syntax and parameter definitions.

SQL

```
SELECT ai_query(
  "<model name>", question
) FROM (VALUES ('what is MLflow?'), ('how does MLflow work?')) AS
t(question);
```

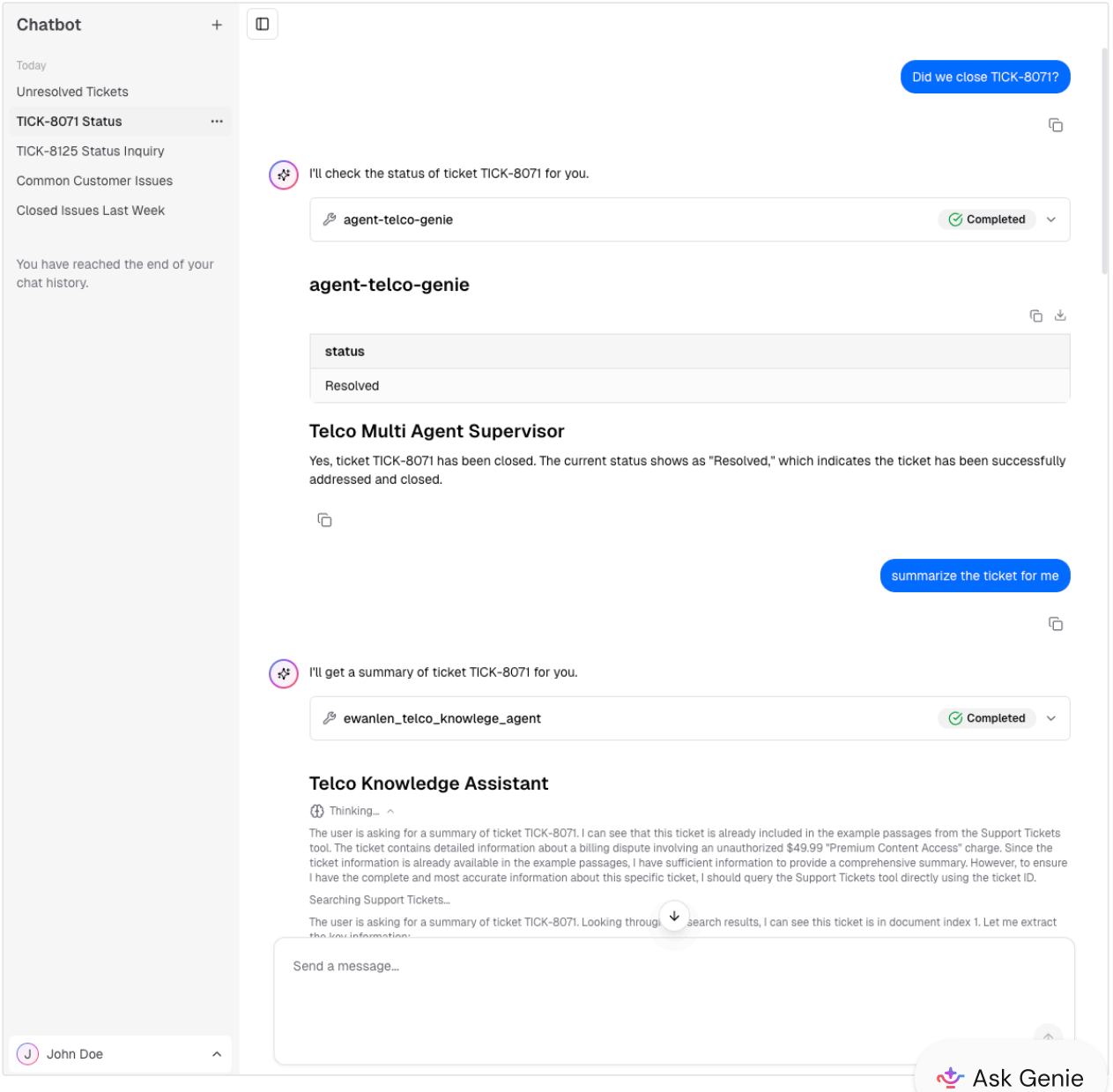
# Next steps

Monitor GenAI apps in production

*Last updated on **May 6, 2026***

# Build and share a chat UI with Databricks Apps

Use Databricks Apps to build and deploy a chat UI for your agent. Agent app templates include this chat UI with no additional setup. Use this page to customize the template UI or to add a chat UI to an agent deployed without a template.



# Requirements

This chat UI works with any Databricks agent, except legacy schemas, including:

- Agents deployed on Apps
- Agents deployed on Model Serving with the **Chat** or **Responses** task type
- Knowledge Assistant
- Supervisor Agent
- Foundation model endpoints with **Chat** task type

You must have the following development tools:

- **NPM CLI**: Required for local development. See GitHub - NPM CLI
- **Databricks CLI**: Required for authentication, see the installation guide.

  i. Install the Databricks CLI.

  ii. Set your profile name:

  ```bash
  export DATABRICKS_CONFIG_PROFILE='your_profile_name'
  ```

  iii. Configure authentication:

  ```bash
  databricks auth login --profile "$DATABRICKS_CONFIG_PROFILE"
  ```

# Example chat application

The example app, e2e-chatbot-app-next uses NextJS, React, and AI SDK to build a production-ready chat interface.

See the project README.md for detailed instructions on how to use the template.



The example app demonstrates the following:

- **Streaming output**: Displays agent responses as they generate with automatic fallback to non-streaming mode
- **Tool calls**: Renders tool calls for agents authored using Agent Framework best practices
- **Databricks Agent and Foundation Model integration**: Direct connection to Foundation Models, Databricks Agent serving endpoints, Knowledge Assistant, and Supervisor Agent.
- **Databricks authentication**: Uses Databricks authentication to identify end users of the chat app and securely manage their conversations.
- **Persistent chat history**: Stores conversations in Databricks Lakebase (Postgres) with full governance

Open the following sections to enable optional features:

Enable chat history >

Enable user feedback >

Host multiple apps on the same database instance >

Enable user authorization (Public Preview) >

# Share the app

Grant users permission to view the app (see Configure permissions for a Databricks app), then share the app URL.

# Known limitations

- No support for image or other multi-modal inputs
- This app only supports Databricks CLI auth (local development) and service principal auth (deployed apps) only. PAT, Azure managed identities, and other mechanisms are not supported.

- Unity Catalog function scopes are not supported for user authorization.

# Streamlit agent chat app

The previous Streamlit template, e2e-chatbot-app, is still available but lacks the production features of e2e-chatbot-app-next.

*Last updated on **May 6, 2026***

# Debug a deployed AI agent

This page covers how to debug common issues with AI agents deployed on Databricks.

Go to:



Most debugging sections on this page apply to agents deployed to Databricks Apps. However, you can also find debugging information for agents deployed on Model Serving (legacy) using the tab selectors.

## Author agents using best practices

Use the following best practices when authoring agents:

- **Enable MLflow tracing**: Follow the best practices in Author an AI agent and deploy it on Databricks Apps. Enable MLflow trace autologging to make your agents easier to debug.
- **Document tools clearly**: Clear tool and parameter descriptions ensure your agent understands your tools and uses them appropriately. See Improve tool-calling with clear documentation.
- **Add timeouts and token limits to LLM calls**: Add timeouts and token limits to the LLM calls in your code to avoid delays caused by long-running steps.

    - If your agent uses the OpenAI client to query a Databricks LLM serving endpoint, set custom timeouts on the serving endpoint calls as needed.
- **Validate configuration before deployment**: Run `databricks bundle validate` before you deploy to catch YAML configuration issues early. This helps identify mismatched resource references, invalid permissions, and syntax errors.
- **Test locally first**: Use local development to catch issues before you deploy. Start your agent server locally, test with sample requests, and verify that MLflow traces appear correctly before you deploy to Databricks Apps.

# Debug local development issues

Test your agent locally to identify issues before deployment.

Before you run your agent locally, verify that your environment is configured correctly:

1. **Check Databricks CLI version**: Run `databricks -v` to verify that you have version 0.283.0 or later.
2. **Verify CLI profiles**: Run `databricks auth profiles` to see the configured authentication profiles.
3. **Validate environment configuration**: Check that your `.env` file contains the required variables, especially `MLFLOW_TRACKING_URI`, which must use the format `databricks://PROFILE_NAME` to include your CLI profile.

## Common local development errors

| Error | Cause | Solution |
| --- | --- | --- |
| `The provided MLFLOW_EXPERIMENT_ID does not exist` | Wrong tracking URI format or experiment was deleted | Verify that `MLFLOW_TRACKING_URI` uses the `databricks://PROFILE_NAME` format with your CLI profile name |
| `Module not found` | Dependencies not installed | Run `uv sync` to install dependencies |

| Error | Cause | Solution |
| --- | --- | --- |
| `Port already in use` | Another process using the port | Use `--port` flag to specify a different port (e.g., `uv run start-app --port 8001`) |
| Authentication errors when running locally | The environment is not configured | Run the quickstart script or manually configure the `.env` file with your CLI profile |

## Test the agent locally

To test your agent before deployment:

1. Start the agent server locally:

```
Bash

uv run start-app
```

2. In another terminal, send a test request:

```
Bash

curl -X POST http://localhost:8000/invocations \
  -H "Content-Type: application/json" \
  -d '{"input": [{"role": "user", "content": "hello"}]}'
```

3. View MLflow traces in the Databricks UI to verify your agent is logging traces correctly.

# Debug configuration issues

Configuration errors in `databricks.yml` and `app.yaml` are common sources of deployment failures.

# Validate the Declarative Automation Bundles configuration

Validate the Declarative Automation Bundles configuration before deploying the app:

```
Bash

databricks bundle validate
```

This command checks your configuration for:

- YAML syntax errors
- Missing required fields
- Not valid resource references
- Permission configuration issues

# Common configuration mismatches

| Configuration point | Rule | How to debug |
| --- | --- | --- |
| `valueFrom` references in `app.yaml` | Must exactly match a resource `name` in `databricks.yml` | Search for the exact string in bo |
| App name | Must start with the `agent-` prefix (e.g., `agent-data-analyst`) | Check the `name` field under `res` |
| Genie Space ID | Must be the 32-character hex string from the Genie URL | Extract from the URL path: `https://workspace.cloud.data` |
| Unity Catalog function reference | Must use format `catalog.schema.function_name` | Verify the function exists using `functions list` |

| Configuration point | Rule | How to debug |
| --- | --- | --- |
| Lakebase instance reference | Must use `value` (not `valueFrom`) in the `app.yaml` file | The instance name is a literal st |

# Debug deployment issues

Agents deployed to Apps | Agents on Model Serving (legacy)

- App already exists error
- App not updating after deployment
- View deployment status and logs

# Debug runtime errors

Agents deployed to Apps | Agents on Model Serving (legacy)

Use app logs and request testing to identify issues with your deployed agent.

## Analyze app logs

View real-time logs from your deployed app:

Bash

```
databricks apps logs <app-name> --follow
```

Ask Genie

Look for:

- Stack traces indicating code errors
- Permission denied messages for resources
- Connection errors to external services
- Timeout messages

## Common runtime errors

| Error | Cause | Solution |
|---|---|---|
| 302 redirect when querying app | Using Personal Access Token instead of OAuth | Get an OAuth token with `databricks auth token` |
| Agent not using available tools | Tools not returned from MCP client | Verify the MCP server URL is correct and the resource has proper permissions in `databricks.yml` |
| Streaming response breaks mid-response | Connection timeout | Increase the `CHAT_PROXY_TIMEOUT_SECONDS` environment variable in `app.yaml` |
| Agent returning "Memory not available" | Missing `user_id` in request | Pass `custom_inputs.user_id` in the request payload |
| Empty or error responses despite 200 status | Error occurred within streamed response | Check the actual stream content and app logs, not just the HTTP status code |

# Debug authentication errors

OAuth token authentication required

Resource permission errors

Custom MCP server permissions

# Debug memory and storage issues

For agents using Lakebase for memory storage, the following issues are common:

| Error | Cause | Solution |
|---|---|---|
| `relation 'store' does not exist` | Memory tables not initialized | Run `await store.setup()` locally before deploying to create required tables |
| `Unable to resolve :re[LKB] instance` | Wrong instance name or incorrect configuration | Verify `LAKEBASE_INSTANCE_NAME` uses `value` (not `valueFrom`) in `app.yaml` and matches the `instance_name` in `databricks.yml` |
| `permission denied for table store` | Missing Lakebase permissions | Add a `database` resource in `databricks.yml` with `permission: 'CAN_CONNECT_AND_CREATE'` |
| Memory not persisting across conversations | Different `user_id` per request | Ensure you pass a consistent `user_id` in `custom_inputs` for each user |

Example Lakebase resource configuration:

YAML

```
resources:
  apps:
    my_agent:
      resources:
        - name: 'memory_database'
          database:
            instance_name: '<lakebase-instance-name>'
            database_name: 'postgres'
            permission: 'CAN_CONNECT_AND_CREATE'
```

Before deploying an agent with memory, initialize the tables locally:

Python

```python
import asyncio
from databricks_langchain import AsyncDatabricksStore

async def setup_memory():
    async with AsyncDatabricksStore(
        instance_name='your-lakebase-instance',
        embedding_endpoint='databricks-gte-large-en',
        embedding_dims=1024,
    ) as store:
        await store.setup()

asyncio.run(setup_memory())
```

**this page**

**as this page helpful?**

Yes | No

Send feedback

Ask Genie